## Learning Objectives

- Learn that the choice of a data structure should depend on the operations we want to perform efficiently.
- Design a specialized string representation that makes equality comparisons particularly cheap.

## Keeping Your Work

Keep the code you develop for the compiler project in a `not-abc` directory. If you are already familiar with Git, you may instead create a `not-abc` repository and keep your work there. We will introduce Git in a later worksheet, so **no Git experience is required at this point**.

## Motivation

In Worksheet 9, we implemented a symbol table for our compiler. For simplicity, we represented identifiers using

```
type String: array[42] of char;
```

This worked, but it was deliberately only a temporary solution.
Why `42` characters? An identifier may need only three characters — or it may need more than `42`. If we choose a large fixed size, we waste memory. If we choose a small fixed size, some identifiers no longer fit.
There is another issue that is particularly important for a symbol table: string comparison. Looking up an identifier requires us to compare its name with names already stored in the table. With ordinary null-terminated strings, equality cannot in general be determined by comparing two pointers. Instead, the characters have to be inspected.
In the preparation sheet, we investigated how common string operations work. We will now develop a representation that is better suited to our particular use case.
The goal is not to design a general-purpose string type that can do everything reasonably well. Instead, we will identify an operation that matters particularly much for our application and choose a representation that makes this operation very efficient.

**Remark:** There are other important use cases with very different requirements. For example, a lexer has to construct strings incrementally while reading characters. A dynamically growing string based on `realloc` would be useful for that purpose. We will need something like this later, but it is deliberately not the data structure we develop in this worksheet.

## Step 1 — Unique Strings [In class]

In the previous step, we saw that comparing two null-terminated strings for equality may require comparing their characters one by one. This matters for our compiler. Recall the symbol table from Worksheet 9: when searching for an identifier, we traversed a linked list and used `strcmp` to compare the identifier we were looking for with the identifier stored in each node.
We will now develop a specialized string representation that makes equality comparison particularly cheap.

The idea is simple: we store each distinct string only once. If the same string is requested again, we return a pointer to the already existing copy. We call such strings unique strings, or `UStr` for short.

Consequently, for two `UStr` values `s1` and `s2`,

```
s1 == s2
```

if and only if they represent the same sequence of characters. Equality comparison therefore requires only a pointer comparison. We no longer have to compare the characters one by one with `strcmp`.

There is a price to pay: creating a `UStr` is more expensive than simply storing a pointer to an existing string. `UStrCreate` has to determine whether an equal string has already been stored and, if necessary, allocate memory and copy the string. This is a deliberate trade-off. For our application, strings are created relatively rarely but compared for equality very frequently.

We are not trying to design a general-purpose string type that performs every possible operation efficiently. We are choosing a representation that is particularly well suited to an important operation in our compiler.

© Michael C. Lehn | CC BY-SA 4.0

Let us first specify how our new string representation should be used. The header file `ustr.hdr` contains only the following declarations:

```
type UStr: readonly char;

extern fn UStrCreate(s: ->readonly char): ->UStr;
```

A `UStr` is therefore still represented by a pointer to a null-terminated sequence of characters. The important difference is the guarantee provided by `UStrCreate`: two `UStr` pointers are equal if and only if the strings they represent are equal.

There is one additional convention: `UStrCreate(nullptr)` is valid and returns `nullptr`. This allows an absent string to remain represented by nullptr.

Before implementing `UStrCreate`, we write a small test program that describes the behavior we expect from it.

```
 1 @ <stdio.hdr>
 2 @ "ustr.hdr"
 3
 4 fn main()
 5 {
 6     local a1: array[] of char = "hallo";
 7     local a2: array[] of char = "hello";
 8     local a3: array[] of char = "hallo";
 9
10     printf("a1 = %p '%s'\n", a1, a1);
11     printf("a2 = %p '%s'\n", a2, a2);
12     printf("a3 = %p '%s'\n", a3, a3);
13
14     local s: array[] of ->UStr = {
15         UStrCreate(a1),
16         UStrCreate(a2),
17         UStrCreate(a3),
18     };
19
20     printf("check addresses of unique strings:\n");
21     for (local i := 0; i < sizeof(s) / sizeof(s[0]); ++i) {
22         printf("s[%d] = %p '%s'\n", i, s[i], s[i]);
23     }
24
25     printf("check fast compare of strings:\n");
26     for (local i := 0; i < sizeof(s) / sizeof(s[0]); ++i) {
27         for (local j := 0; j < sizeof(s) / sizeof(s[0]); ++j) {
28             printf("s[%d] == s[%d] gives: %d\n", i, j, s[i] == s[j]);
29         }
30     }
31 }
```

The arrays `a1`, `a2`, and `a3` contain three ordinary null-terminated strings. Although `a1` and `a3` contain the same sequence of characters, they occupy different memory locations. Their addresses are therefore different.

`UStrCreate` converts these ordinary strings into unique strings. Since `"hallo"` must be stored only once, `s[0]` and `s[2]` must point to the same memory location, whereas `s[1]` must point somewhere else.
The final part of the program exploits exactly this property: equality of two unique strings can be tested with `==`.

### Tasks

1. Create the header file `ustr.hdr` and the test program `xtest_ustr.abc` shown above. Try to build the program with

   ```
   abc xtest_ustr.abc
   ```

   At this point, the build should fail at the linking stage because `UStrCreate` has been declared but not yet implemented.

© Michael C. Lehn | CC BY-SA 4.0

2. **Implementing `UStrCreate`.** The following code for is `ustr.abc` provided:

```
@ <string.hdr>
@ <stdlib.hdr>
@ "ustr.hdr"

struct UStrNode
{
    next: ->UStrNode;
    data: array[1] of char;
};

global list: -> UStrNode;

fn UStrCreate(s: -> readonly char): ->UStr
{
    if (!s) {
        return nullptr;
    }
    local n: -> UStrNode = list;
    // TODO 2: Check if string is already in list
    local len: size_t = strlen(s);
    n = malloc(sizeof(*n) + len);
    // TODO 1: Copy string into node and prepend node to list
    return list->data;
}
```

   Type the code above into `ustr.abc`. The program should now compile with

```
abc xtest_ustr.abc ustr.abc
```

   and link successfully. However, running it will still result in a segmentation fault because function `UStrCreate` is not yet complete. In the following tasks, we will complete the implementation in two steps. First, we will allocate and store strings in a linked list. After that works, we will make sure that each distinct string is stored only once.

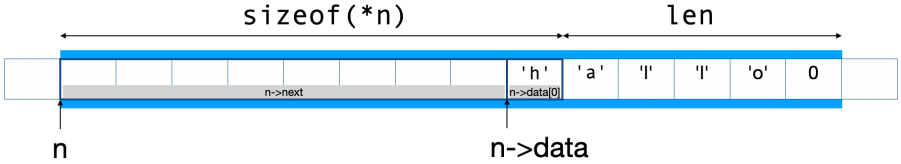
3. **Storing strings in the list.** Before implementing `TODO 1`, let us take a closer look at the data structure `UStrNode`. The member `next` should look familiar from the linked lists used in Worksheet 9. It points to the next node in the list. The second member is more unusual: `data` is an array containing only a single character. The structure therefore requires at least enough memory for one pointer and one character. Its actual size may be larger. We will discuss the reason for this later; it is not important for what follows.

   When creating a node, we deliberately allocate additional memory:

```
n = malloc(sizeof(*n) + len);
```

   The allocated memory block is large enough to hold
   - the pointer stored in `next`,
   - all `len` characters of the string, and
   - the terminating null character.

   The important point is that the allocated memory block is large enough to hold the pointer `next`, all characters of the string, and the terminating null character. Later, when we discuss **alignment** and **padding**, we will see that the block may actually contain some unused bytes. For now, it is sufficient to know that the block cannot be too small.
   Starting at the address `n->data`, we can therefore use `strcpy` to copy the complete null-terminated string into the newly allocated node.




   Now complete `TODO 1`. Copy the string into the new node and prepend the node to list.
   Compile and run the test program again. It should now run without errors. However, strings with equal contents are still stored in different nodes. In particular, check the addresses returned for the two occurrences of `"hallo"`.

© Michael C. Lehn | CC BY-SA 4.0

4. **Making the strings unique.** Complete `TODO 2`. Before allocating a new node, traverse the list and use `strcmp` to check whether the string is already stored in one of its nodes.
   If an equal string is found, return `n->data` immediately. Only if the complete list has been searched without finding the string should a new node be allocated and prepended to the list.
   Compile and run the test program again. Verify that the two occurrences of `"hallo"` now have the same address, while `"hello"` has a different address. Consequently,

   ```
   s[0] == s[2]
   ```

   must evaluate to true.

### Step 2 — Bringing Back Our Symbol Table (Preparation) [In class]

In Worksheet 9, we implemented a symbol table using a linked list. We will now bring this implementation into our compiler project and replace its temporary string representation with our new `UStr` representation.

Copy the following files from Session 9 into your `not-abc` directory:

- `symtab.hdr`
- `symtab.abc`
- `xtest_symtab.abc`

Do not copy `string.hdr`. The old `symtab.hdr` still includes this file because our symbol table used the temporary

```
type String: array[42] of char;
```

representation introduced in Worksheet 9. Try to build the old test program:

```
abc xtest_symtab.abc symtab.abc
```

The build should fail because `string.hdr` cannot be found. This is intentional. Do not fix the problem by copying `string.hdr`.

Instead, we will make a small but important change to our symbol table: identifiers will no longer be represented by fixed-size arrays. In the next step, we will replace this old representation with `UStr`.

### Step 3 — Changing the Symbol Table Interface [In class]

Our symbol table is still using the temporary string representation from Worksheet 9. Before changing its implementation, we first change its interface and the corresponding test program. Open `symtab.hdr`. It still includes `string.hdr` and uses String to represent identifiers. We want identifiers stored in the symbol table to be unique strings instead.

#### Tasks

1. Open `symtab.hdr`. It still includes `string.hdr` and uses `String` to represent identifiers. We want identifiers stored in the symbol table to be unique strings instead.
   (a) Include `ustr.hdr`. instead of `string.hdr`.
   (b) Replace type `String` with `->UStr`.
2. Adapt the test program `xtest_symtab.abc` to the new interface. If `xtest_symtab.abc` includes `string.hdr`, remove that include. The string representation used by the symbol table is now provided through `symtab.hdr`, which in turn includes `ustr.hdr`.
   Next, adapt the calls to the functions declared in `symtab.hdr`. In the old test program, string literals were sometimes explicitly converted to the fixed-size `String` type, for example:

   ```
   (String)"bar"
   ```

   We no longer have this type. Instead, create a unique string:

   ```
   UStrCreate("bar")
   ```

   Make the corresponding changes throughout `xtest_symtab.abc` so that all functions from `symtab.hdr` are called with arguments of the correct types.

Finally, compile the test program with `abc xtest_symtab.abc`. This step is complete when only linker errors remain.

© Michael C. Lehn | CC BY-SA 4.0

### Step 4 — Using Unique Strings in the Symbol Table [In class]

We have changed the interface of our symbol table and adapted its test program. What remains is to update the implementation in `symtab.abc`.

Fortunately, this requires only two small changes:

1. Replace the type String by `->UStr` wherever identifiers are stored or passed around.
2. In the helper function isInList, replace the string comparison

   ```
   !strcmp(list->ident, ident)
   ```

   by a comparison of the two pointers using `==`.
   **Remember why this works:** `UStrCreate` guarantees that equal strings are represented by the same address. Comparing identifiers therefore no longer requires comparing their characters one by one.
   Now build the complete test program:

   ```
   abc xtest_symtab.abc symtab.abc ustr.abc
   ```

   It should compile, link, and run without errors. In particular, an identifier must not occur more than once in the symbol table.
   Finally, study `symtab.abc` and `xtest_symtab.abc` once more. Make sure you understand how the test program checks the behavior of the symbol table. Where would its output change if comparing identifiers with `==` did not work correctly?

### Step 5 — Changing the Lexer Interface [In class]

We will now start adapting our lexer to use unique strings. The files `lexer.hdr`, `lexer.abc`, and `xtest_lexer.abc` should already be in your not-abc directory from Session 11.
For now, we only change the interface in `lexer.hdr`:

1. Remove the old type declaration

   ```
   type TokenVal: array[13] of char;
   ```

2. At the top of `lexer.hdr`, include the interface for unique strings:

   ```
   @ "Ustr.hdr"
   ```

3. Change the type of the val member in Token from TokenVal to ->UStr. The declaration should now look like this:

   ```
   struct Token
   {
       kind:   TokenKind;
       pos:    struct Pos {
                   row, col: unsigned;
               };
       val:    ->UStr;
   };
   ```

Finally, compile the test program with `abc xtest_lexer.abc`.

**This step is complete when only linker errors remain.**

### Looking Ahead

There is one problem left. While scanning a token, the lexer reads its characters one by one. We therefore still need a place where we can successively collect these characters.

A `UStr` is not suitable for this: once created, a unique string can not be modified. In the next step, we will therefore use an internal `char` array in `lexer.abc` as a temporary buffer. While scanning a token, its characters are collected in this buffer. Once the token is complete, we create a unique string with `UStrCreate` and assign the resulting `->UStr` to `token.val`.

In other words, the temporary buffer is used while constructing a token; `token.val` represents the finished token value.

© Michael C. Lehn | CC BY-SA 4.0

### Step 6 — Adapting the Lexer Implementation

We now adapt the implementation in `lexer.abc`. The basic idea is to keep the construction of a token separate from its final representation as a `UStr`.
First, add an internal array for collecting the characters of the current token. Place it near the top of `lexer.abc`, before `nextCh`:

```
global internalVal: array[13] of char;
```

Next, modify `nextCh` so that it stores the characters belonging to the current token in the internal array `internalVal` instead of `token.val`. This is essentially a search-and-replace operation. Afterwards, `nextCh` should look like this:

```
fn nextCh()
{
    if (updateVal) {
        assert(lengthVal + 1 < sizeof(internalVal) / sizeof(internalVal[0]));
        internalVal[lengthVal++] = ch;
        internalVal[lengthVal] = 0;
    }
    ch = getchar();
    if (ch == '\n') {
        ++currentPos.row;
        currentPos.col = 0;
    } else {
        ++currentPos.col;
    }
}
```

Now rename the existing function `getToken` to `getToken_`. This makes it an internal function that is used only within the translation unit `lexer.abc`.
Finally, implement a new `getToken` as a wrapper around `getToken_`:

```
fn getToken(): TokenKind
{
    getToken_();
    token.val = UStrCreate(internalVal);
    return token.kind;
}
```

The wrapper first calls `getToken_`, which scans the next token and collects all its characters in `internalVal`. Once the complete token has been recognized, `UStrCreate` turns this temporary character sequence into a unique string and assigns it to `token.val`.
The interface of the lexer therefore remains simple: `getToken` still returns the `TokenKind` set by `getToken_`, while `token.val` contains the corresponding unique string.

**Note:** `internalVal` is for constructing a token; `token.val` is for representing the finished token.

Finally, build the lexer test program:

```
abc xtest_lexer.abc lexer.abc ustr.abc
```

It should now compile and link without errors. Run the test program and verify that the lexer still recognizes and reports the tokens correctly.

### Looking Ahead

The standard library header stdlib.hdr also declares the function

```
extern fn realloc(ptr: ->void, size: size_t): ->void;
```

`realloc` changes the size of an existing dynamically allocated memory block pointed to by `ptr`. If possible, the block is resized in place. Otherwise, a new block is allocated, the existing data is copied to it, and the old block is released. The returned pointer therefore may differ from `ptr`.

We will use `realloc` in the next step to build a data structure that can grow dynamically as needed.

© Michael C. Lehn | CC BY-SA 4.0

### Step 7 — Letting Tokens Grow

Our lexer can still store at most 12 characters in a token. Let us remove this restriction. Since we will use `realloc`, first include the standard library header at the top of `lexer.abc`:

```
@ <stdlib.hdr>
```

Then replace the fixed-size array

```
global internalVal: array[13] of char;
```

by

```
global internalVal: ->char;
global internalValCapacity: u64;
```

`internalVal` points to a dynamically allocated block of characters used to construct the current token, while `internalValCapacity` stores the size of this block. Since both are uninitialized global variables, they reside in the BSS segment and are initially zero (`nullptr` and `0`, respectively).

Now modify `nextCh`:

```
fn nextCh()
{
    if (updateVal) {
        if (lengthVal + 1 > internalValCapacity) {
            internalValCapacity = 2 * internalValCapacity + 1;
            internalVal = realloc(internalVal, internalValCapacity);
        }
        internalVal[lengthVal++] = ch;
        internalVal[lengthVal] = 0;
    } else {
        lengthVal = 0;
    }
    ch = getchar();
    if (ch == '\n') {
        ++currentPos.row;
        currentPos.col = 0;
    } else {
        ++currentPos.col;
    }
}
```

Whenever there is not enough space for the next character and the terminating null character, the buffer grows. Its capacity changes from `0` to `1`, then to `3`, `7`, `15`, and so on. `realloc` preserves the characters already collected, so scanning can simply continue with the larger buffer.
We have therefore removed the fixed limit on the length of a token: `internalVal` grows whenever more space is needed.

Finally, rebuild the lexer test program:

```
abc xtest_lexer.abc lexer.abc ustr.abc
```

Test it thoroughly. In particular, try identifiers that are much longer than the previous 12-character limit and verify that they are recognized correctly.

© Michael C. Lehn | CC BY-SA 4.0